

Autonomous Navigation with Nav2 & TurtleBot3

This repository contains the implementation of autonomous navigation for a TurtleBot3 mobile robot in a ROS 2 Humble environment. The project demonstrates map-based localization using AMCL and multi-waypoint mission planning.

🚩 Project Objective

The goal of this lab was to integrate the full autonomous navigation pipeline:

- * Load a pre-saved occupancy map.
- * Localize the robot within the map using Adaptive Monte Carlo Localization (AMCL).
- * Execute complex missions by navigating through a series of defined waypoints.

🛠️ Prerequisites

- * **OS:** Ubuntu 22.04 with ROS 2 Humble.
- * **Simulation:** Gazebo and RViz.
- * **Packages:** `nav2\_bringup`, `turtlebot3\_gazebo`, and `turtlebot3\_navigation2`[cite: 1].
- * **Hardware Model:** TurtleBot3 Burger[cite: 1].

📁 Repository Structure

- * `/week7_pkg``: The ROS 2 Python package containing navigation nodes[cite: 1].
- * `/week7_pkg/waypoint_navigator.py``: Python node to send a sequence of goals[cite: 1].
- * `/screenshots``: Visual evidence of localization, costmaps, and mission success[cite: 1].
- * `/maps``: The `.pgm`` and `.yaml`` files used for navigation[cite: 1].

🚀 How to Run

1. Launch Simulation

Open a terminal and start the Gazebo environment:

```
``bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

2. Launch Navigation

In a second terminal, start the Nav2 stack with your map:

Bash

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True
map:=$HOME/maps/my_map.yaml
```

3. Localization

- In RViz, use the **2D Pose Estimate** tool to set the robot's initial position[cite: 1].
- Verify that the AMCL particle cloud (red arrows) aligns with the robot[cite: 1].

4. Execute Waypoint Mission

Run the custom navigator node:

Bash

```
``
```

python3 waypoint_navigator.py



Results & Observations

- **Mission Success:** The robot successfully reached all 5 defined waypoints in sequence[cite: 1].
- **Costmaps:** Observed inflation layers in the local costmap that prevented collisions with dynamic obstacles[cite: 1].
- **Recovery:** When blocked, the robot utilized recovery behaviors like replanning to bypass obstacles[cite: 1].